Q1. Relational Algebra Operations

Relational algebra is a procedural query language used to retrieve and manipulate data stored in relational databases. It consists of different operations such as selection, projection, union, set difference, Cartesian product, and join.

Given Relations:

- **EMPLOYEE** (Emp_ID, Emp_Name, Salary, Dept_ID)
- **DEPARTMENT** (Dept_ID, Dept_Name, Location)

1. Selection (σ)

Definition: Selection is a relational algebra operation used to retrieve rows (tuples) from a relation that satisfy a specified condition. It filters the records based on the given condition.

Operation:

σ_{Dept_ID = 10}(EMPLOYEE)

Explanation: This operation retrieves all employees who belong to department 10. Only the rows satisfying the condition are selected, while the remaining rows are excluded from the result.

2. Projection (π)

Definition: Projection is a relational algebra operation used to retrieve specific columns (attributes) from a relation. It displays only the required attributes and removes duplicate tuples.

Operation:

π_{Dept_ID, Dept_Name}(DEPARTMENT)

Explanation: This operation retrieves only the department ID and department name from the DEPARTMENT relation. Other attributes, such as Location, are excluded from the result.

3. Union (∪)

Definition: Union is a relational algebra operation that combines tuples from two compatible relations into a single relation. Both relations must have the same number of attributes and compatible corresponding domains. Duplicate tuples are removed.

Operation:

σ_{Dept_ID = 10}(EMPLOYEE) ∪ σ_{Dept_ID = 20}(EMPLOYEE)

Explanation: This operation retrieves employees who belong to either department 10 or department 20. The two selected relations have the same attributes, so they can be combined using the union operation.

4. Set Difference (−)

Definition: Set difference is a relational algebra operation that returns tuples present in the first relation but not present in the second relation. Both relations must have compatible schemas.

Operation:

EMPLOYEE − σ_{Dept_ID = 10}(EMPLOYEE)

Explanation: This operation retrieves employees who do not belong to department 10. It removes all employees belonging to department 10 from the EMPLOYEE relation.

5. Cartesian Product (×)

Definition: Cartesian product is a relational algebra operation that combines every tuple of one relation with every tuple of another relation. If the first relation contains m tuples and the second contains n tuples, the resulting relation contains m × n tuples.

Operation:

EMPLOYEE × DEPARTMENT

Explanation: This operation combines every employee record with every department record, producing all possible combinations. It does not require a matching condition between the two relations.

6. Join (⋈)

Definition: Join is a relational algebra operation that combines related tuples from two relations based on a specified condition. It is used to retrieve related information stored in different relations.

Operation:

EMPLOYEE ⋈_{EMPLOYEE.Dept_ID = DEPARTMENT.Dept_ID} DEPARTMENT

Explanation: This operation combines employee records with their corresponding department records by matching Dept_ID in both relations. It retrieves employee details along with department names and locations.

Conclusion

Relational algebra provides a set of operations for retrieving and manipulating data in relational databases. Selection retrieves specific rows, projection retrieves required columns, union combines compatible relations, set difference identifies tuples not present in another relation, Cartesian product generates all possible combinations, and join combines related information from multiple relations.

Q2)

```
mysql> CREATE TABLE DEPARTMENT_Q2 (
    ->     DeptID INT PRIMARY KEY,
    ->     DeptName VARCHAR(100) NOT NULL UNIQUE
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE STUDENT_Q2 (
    ->     SID INT PRIMARY KEY,
    ->     Name VARCHAR(100) NOT NULL UNIQUE,
    ->     DOB DATE NOT NULL DEFAULT '2000-01-01',
    ->     DeptID INT,
    ->     CHECK (DOB >= '1900-01-01'),
    ->     FOREIGN KEY (DeptID) REFERENCES DEPARTMENT_Q2(DeptID)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> DESCRIBE DEPARTMENT_Q2;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| DeptID   | int          | NO   | PRI | NULL    |       |
| DeptName | varchar(100) | NO   | UNI | NULL    |       |
+----------+--------------+------+-----+---------+-------+
2 rows in set (0.00 sec)

mysql> DESCRIBE STUDENT_Q2;
+--------+--------------+------+-----+------------+-------+
| Field  | Type         | Null | Key | Default    | Extra |
+--------+--------------+------+-----+------------+-------+
| SID    | int          | NO   | PRI | NULL       |       |
| Name   | varchar(100) | NO   | UNI | NULL       |       |
| DOB    | date         | NO   |     | 2000-01-01 |       |
| DeptID | int          | YES  | MUL | NULL       |       |
+--------+--------------+------+-----+------------+-------+
4 rows in set (0.00 sec)
```

Q3)

```
mysql> CREATE TABLE SALES_Q3 (
    ->     Sale_ID INT PRIMARY KEY,
    ->     Product_ID INT NOT NULL,
    ->     Customer_ID INT NOT NULL,
    ->     Quantity INT NOT NULL,
    ->     Unit_Price DECIMAL(10,2) NOT NULL,
    ->     Sale_Date DATE NOT NULL
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO SALES_Q3 VALUES
    -> (1, 101, 1, 2, 100.00, '2026-01-10'),
    -> (2, 101, 2, 3, 100.00, '2026-01-11'),
    -> (3, 102, 1, 5, 50.00, '2026-01-12'),
    -> (4, 102, 3, 2, 50.00, '2026-01-13'),
    -> (5, 103, 2, 10, 25.00, '2026-01-14'),
    -> (6, 103, 4, 4, 25.00, '2026-01-15'),
    -> (7, 104, 3, 1, 500.00, '2026-01-16'),
    -> (8, 104, 5, 2, 500.00, '2026-01-17');
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM SALES_Q3;
+---------+------------+-------------+----------+------------+------------+
| Sale_ID | Product_ID | Customer_ID | Quantity | Unit_Price | Sale_Date  |
+---------+------------+-------------+----------+------------+------------+
|       1 |        101 |           1 |        2 |     100.00 | 2026-01-10 |
|       2 |        101 |           2 |        3 |     100.00 | 2026-01-11 |
|       3 |        102 |           1 |        5 |      50.00 | 2026-01-12 |
|       4 |        102 |           3 |        2 |      50.00 | 2026-01-13 |
|       5 |        103 |           2 |       10 |      25.00 | 2026-01-14 |
|       6 |        103 |           4 |        4 |      25.00 | 2026-01-15 |
|       7 |        104 |           3 |        1 |     500.00 | 2026-01-16 |
|       8 |        104 |           5 |        2 |     500.00 | 2026-01-17 |
+---------+------------+-------------+----------+------------+------------+
8 rows in set (0.00 sec)
```

```
mysql> SELECT
    ->     Product_ID,
    ->     COUNT(*) AS Total_Sales,
    ->     SUM(Quantity) AS Total_Quantity,
    ->     SUM(Quantity * Unit_Price) AS Total_Revenue,
    ->     AVG(Unit_Price) AS Average_Price,
    ->     MAX(Unit_Price) AS Maximum_Price,
    ->     MIN(Unit_Price) AS Minimum_Price
    -> FROM SALES_Q3
    -> GROUP BY Product_ID;
+------------+-------------+----------------+---------------+---------------+---------------+---------------+
| Product_ID | Total_Sales | Total_Quantity | Total_Revenue | Average_Price | Maximum_Price | Minimum_Price |
+------------+-------------+----------------+---------------+---------------+---------------+---------------+
|        101 |           2 |              5 |        500.00 |    100.000000 |        100.00 |        100.00 |
|        102 |           2 |              7 |        350.00 |     50.000000 |         50.00 |         50.00 |
|        103 |           2 |             14 |        350.00 |     25.000000 |         25.00 |         25.00 |
|        104 |           2 |              3 |       1500.00 |    500.000000 |        500.00 |        500.00 |
+------------+-------------+----------------+---------------+---------------+---------------+---------------+
4 rows in set (0.01 sec)

mysql> SELECT
    ->     Product_ID,
    ->     SUM(Quantity * Unit_Price) AS Total_Revenue
    -> FROM SALES_Q3
    -> GROUP BY Product_ID
    -> HAVING SUM(Quantity * Unit_Price) > 400;
+------------+---------------+
| Product_ID | Total_Revenue |
+------------+---------------+
|        101 |        500.00 |
|        104 |       1500.00 |
+------------+---------------+
2 rows in set (0.00 sec)

mysql> SELECT
    ->     Product_ID,
    ->     SUM(Quantity) AS Total_Quantity
    -> FROM SALES_Q3
    -> GROUP BY Product_ID
    -> HAVING SUM(Quantity) > 5;
+------------+----------------+
```

```
mysql> SELECT
    ->     Product_ID,
    ->     SUM(Quantity) AS Total_Quantity
    -> FROM SALES_Q3
    -> GROUP BY Product_ID
    -> HAVING SUM(Quantity) > 5;
+------------+----------------+
| Product_ID | Total_Quantity |
+------------+----------------+
|        102 |              7 |
|        103 |             14 |
+------------+----------------+
2 rows in set (0.00 sec)
```

Q4)

```
mysql> CREATE TABLE EMPLOYEE_Q4 (
    ->     Emp_ID INT PRIMARY KEY,
    ->     Emp_Name VARCHAR(50) NOT NULL,
    ->     Salary DECIMAL(10,2) NOT NULL,
    ->     Dept_ID INT NOT NULL
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO EMPLOYEE_Q4 VALUES
    -> (1, 'Arun', 50000, 10),
    -> (2, 'Bala', 40000, 10),
    -> (3, 'Charan', 60000, 10),
    -> (4, 'Divya', 30000, 20),
    -> (5, 'Esha', 45000, 20),
    -> (6, 'Farhan', 35000, 20);
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT Emp_ID, Emp_Name, Salary, Dept_ID
    -> FROM EMPLOYEE_Q4 E1
    -> WHERE Salary > (
    ->     SELECT AVG(Salary)
    ->     FROM EMPLOYEE_Q4 E2
    ->     WHERE E2.Dept_ID = E1.Dept_ID
    ->       AND E2.Emp_ID <> E1.Emp_ID
    -> );
+--------+----------+----------+---------+
| Emp_ID | Emp_Name | Salary   | Dept_ID |
+--------+----------+----------+---------+
|      3 | Charan   | 60000.00 |      10 |
|      5 | Esha     | 45000.00 |      20 |
+--------+----------+----------+---------+
2 rows in set (0.01 sec)
```

Q5)

```
mysql> CREATE TABLE EMPLOYEE_Q5 (
    ->     Emp_ID INT PRIMARY KEY,
    ->     Emp_Name VARCHAR(50) NOT NULL,
    ->     Dept_ID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO EMPLOYEE_Q5 VALUES
    -> (1, 'Arun', 10),
    -> (2, 'Bala', 20),
    -> (3, 'Charan', 10),
    -> (4, 'Divya', 30);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE PROJECT_Q5 (
    ->     Project_ID INT PRIMARY KEY,
    ->     Project_Name VARCHAR(50) NOT NULL,
    ->     Emp_ID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO PROJECT_Q5 VALUES
    -> (101, 'Website', 1),
    -> (102, 'Database', 2),
    -> (103, 'Mobile App', 5);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM EMPLOYEE_Q5 E
    -> INNER JOIN PROJECT_Q5 P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+------------+--------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name |
+--------+----------+------------+--------------+
|      1 | Arun     |        101 | Website      |
|      2 | Bala     |        102 | Database     |
+--------+----------+------------+--------------+
2 rows in set (0.00 sec)
```

```
mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM EMPLOYEE_Q5 E
    -> LEFT OUTER JOIN PROJECT_Q5 P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+------------+--------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name |
+--------+----------+------------+--------------+
|      1 | Arun     |        101 | Website      |
|      2 | Bala     |        102 | Database     |
|      3 | Charan   |       NULL | NULL         |
|      4 | Divya    |       NULL | NULL         |
+--------+----------+------------+--------------+
4 rows in set (0.00 sec)

mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM EMPLOYEE_Q5 E
    -> RIGHT OUTER JOIN PROJECT_Q5 P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+------------+--------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name |
+--------+----------+------------+--------------+
|      1 | Arun     |        101 | Website      |
|      2 | Bala     |        102 | Database     |
|   NULL | NULL     |        103 | Mobile App   |
+--------+----------+------------+--------------+
3 rows in set (0.00 sec)
```

```
mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM EMPLOYEE_Q5 E
    -> LEFT JOIN PROJECT_Q5 P
    -> ON E.Emp_ID = P.Emp_ID
    ->
    -> UNION
    ->
    -> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM EMPLOYEE_Q5 E
    -> RIGHT JOIN PROJECT_Q5 P
    -> ON E.Emp_ID = P.Emp_ID;
+--------+----------+------------+--------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name |
+--------+----------+------------+--------------+
|      1 | Arun     |        101 | Website      |
|      2 | Bala     |        102 | Database     |
|      3 | Charan   |       NULL | NULL         |
|      4 | Divya    |       NULL | NULL         |
|   NULL | NULL     |        103 | Mobile App   |
+--------+----------+------------+--------------+
5 rows in set (0.00 sec)

mysql> SELECT E.Emp_ID, E.Emp_Name, P.Project_ID, P.Project_Name
    -> FROM EMPLOYEE_Q5 E
    -> CROSS JOIN PROJECT_Q5 P;
+--------+----------+------------+--------------+
| Emp_ID | Emp_Name | Project_ID | Project_Name |
+--------+----------+------------+--------------+
|      1 | Arun     |        103 | Mobile App   |
|      1 | Arun     |        102 | Database     |
|      1 | Arun     |        101 | Website      |
|      2 | Bala     |        103 | Mobile App   |
|      2 | Bala     |        102 | Database     |
|      2 | Bala     |        101 | Website      |
|      3 | Charan   |        103 | Mobile App   |
|      3 | Charan   |        102 | Database     |
|      3 | Charan   |        101 | Website      |
|      4 | Divya    |        103 | Mobile App   |
|      4 | Divya    |        102 | Database     |
|      4 | Divya    |        101 | Website      |
```